

PROJECT REPORT: College Timetable Generator

Submitted by: KRISHNA RAI

REGISTRATION Number: 25BAI11298

Subject: Python Programming

1. Introduction

The **College Timetable Generator** is a desktop application designed to automate the complex task of scheduling classes. In educational institutions, creating a conflict-free timetable manually is a tedious, error-prone, and time-consuming process. This project leverages Python's computational power to assign courses to time slots while adhering to constraints such as professor availability and room capacity. The application provides a user-friendly Graphical User Interface (GUI) built with tkinter to ensure ease of use for administrative staff.

2. Problem Statement

Current Scenario: Most colleges still rely on manual methods (Excel sheets or paper) to draft schedules. This leads to:

- Double-booking of professors or rooms.
- Inefficient use of time slots.
- Difficulty in modifying the schedule once created.

Proposed Solution: An automated system that accepts course details (Subject, Professor, Room, Frequency) and generates a conflict-free weekly schedule using a randomized greedy algorithm. The system must visualize the output in a clear grid format and allow exporting the data for external use.

3. Functional Requirements

The system implements the following core functional modules:

1. Data Input Module:

- Users can input Subject Name, Professor Name, Room Number, and Lectures per Week.
- Validation to ensure no empty fields are submitted.
- List view to display currently added courses.

2. Scheduling Engine (Processing):

- A logic module that attempts to fit courses into a weekly grid (Mon-Fri, 9 AM - 4 PM).
- **Constraint Checking:** Ensures no double-booking of slots (Room/Professor logic).
- **Randomized Retry:** Uses a shuffling algorithm to find a valid solution if the first attempt fails.

3. Visualization & Output:

- Displays the generated timetable in a grid format with distinct borders.

- Color-coding for different subjects for better readability.
- **CSV Export:** Functionality to save the generated schedule as a .csv file for use in Excel.

4. Non-Functional Requirements

1. **Usability:** The interface is built with standard GUI elements (Buttons, Entry fields) that are intuitive for non-technical users.
2. **Performance:** The scheduling algorithm generates results in under 2 seconds for a standard workload (approx. 20-30 courses).
3. **Reliability:** The system includes error handling (e.g., "Could not fit everything") to inform the user if the input constraints are mathematically impossible.
4. **Portability:** Written in standard Python 3, the application can run on Windows, macOS, or Linux without heavy dependencies.

5. System Architecture

The project follows a simplified **Model-View-Controller (MVC)** pattern:

- **Model:** Data is stored in the self.courses list and self.final_sched dictionary.
- **View:** The tkinter widgets (Treeview, Frame, Label) handle the display of data.
- **Controller:** The App class methods (add_sub, process, try_fit) handle user input and application logic.

6. Design Diagrams

6.1 Use Case Diagram

graph LR

User((Admin))

System[Timetable System]

User -->|Input Details| AddCourse[Add Course Details]

User -->|Click| Generate[Generate Timetable]

User -->|Click| Export[Export to CSV]

Generate -->|Include| CheckConflicts{Check Constraints}

Generate -->|Include| DisplayGrid[Display Grid]

6.2 Workflow Diagram

graph TD

Start([Start App]) --> Input[User Enters Course Details]

Input --> Add{Add More?}

Add -- Yes --> Input

Add -- No --> Gen[Click 'Generate Table']

Gen --> Shuffle[Shuffle Course Blocks]

Shuffle --> SlotLoop[Try Placing in Empty Slots]

SlotLoop --> Check[Check Constraints]

Check -- Conflict --> Retry[Retry / Reshuffle]

Check -- Success --> Save[Save to Schedule]

Save --> Complete{All Assigned?}

Complete -- No --> Retry

Complete -- Yes --> Display[Draw Grid on UI]

Display --> Export[User Clicks 'Save as CSV']

Export --> End([End])

7. Design Decisions & Rationale

1. Language: Python

- *Reason:* Python offers rapid development and built-in support for GUIs (tkinter) and file handling (csv), making it ideal for this project scope.

2. Algorithm: Randomized Greedy Search

- *Reason:* While genetic algorithms are powerful, a randomized greedy approach (shuffling inputs and trying to fit them) is computationally cheaper and sufficiently effective for typical college constraints.

3. UI Library: Tkinter

- *Reason:* It is part of the standard library, requiring no external installation (pip install), which makes the project easy to share and run on college lab computers.

4. Data Structure: Dictionaries

- *Reason:* Using a dictionary with (day_index, time_index) keys provides $O(1)$ access time for checking slot availability, which is faster than iterating through lists.

8. Implementation Details

The project is implemented in a single modular Python file `timetable_final.py`.

- **Key Class: App**
 - `__init__`: Sets up the UI layout using PanedWindow and Grid geometry.
 - `add_sub()`: Collects input from Entry widgets and appends to the self.courses list.
 - `process()`: The main driver that prepares data blocks and triggers the solving loop.
 - `try_fit()`: The core logic that iterates through days/times to find empty spots.
 - `draw_grid()`: Dynamically creates tk.Label widgets with a black background container to simulate table borders.
 - `save_csv()`: Writes the self.final_sched data to a standard CSV format.

Tools Used: VS Code, Python 3.11, Tkinter.

9. Screenshots / Results

(Note: Paste screenshots of your running application here)

1. **Input Screen:** Shows the left panel with course details entered.
2. **Generated Output:** Shows the right panel with the colorful grid and black borders.
3. **Exported CSV:** Shows the resulting file opened in Microsoft Excel.

10. Testing Approach

1. **Unit Testing (Manual):**
 - *Input Validation:* Verified that clicking "Add" with empty fields triggers an error message.
 - *Data Integrity:* Verified that if "Maths" has 3 lectures, exactly 3 blocks appear in the grid.
2. **Constraint Testing:**
 - Added the same professor for two different subjects and verified they were not placed in the same time slot.
3. **Boundary Testing:**
 - Tested with 0 courses (System handles gracefully).
 - Tested with excessive courses (System shows "Could not fit" warning).

11. Challenges Faced

1. **Grid Visualization:** Initially, tkinter does not have a simple grid widget with borders.
 - *Solution:* I used a Frame with a black background and placed labels inside it with `padx=1, pady=1`. This created the visual effect of black border lines.
2. **Constraint Logic:** Ensuring that the loop doesn't get stuck in an infinite cycle if a schedule is impossible.

- *Solution:* Implemented a "Retry Limit" (50 attempts). If it fails 50 times, it alerts the user rather than freezing.
3. **Text Formatting:** Long course names were getting cut off.
- *Solution:* Used wraplength in the label configuration and rowconfigure weight to ensure rows stretch to fill the screen.

12. Learnings & Key Takeaways

- **GUI Geometry Management:** Learned how to use pack, grid, and weight to create responsive layouts that resize with the window.
- **Algorithmic Thinking:** Understood how to map real-world constraints (Room, Time, Professor) into logical checks in code.
- **User Experience:** Realized the importance of visual feedback (colors, borders) in making data-heavy applications readable.

13. Future Enhancements

- **Database Integration:** Replace the in-memory list with SQLite to save courses permanently between sessions.
- **Soft Constraints:** Add logic to prefer morning slots for difficult subjects.
- **Teacher View:** Generate a separate report showing the timetable for a specific teacher.

14. References

1. Python Documentation (Tkinter): <https://docs.python.org/3/library/tkinter.html>
2. Python CSV Module Docs: <https://docs.python.org/3/library/csv.html>
3. StackOverflow discussions on "Tkinter Grid Borders".